

Cost Belongs in the Constraint, Not the Objective: A Held-Out Evaluation of Multi-Criteria Model Selection for Retail Demand Forecasting

Ranjith Kumar N

Department of Computer Science and Engineering
PES University
Bengaluru, Karnataka, India
PES1UG23CS474

Pooja Ammalajeri

Department of Computer Science and Engineering
PES University
Bengaluru, Karnataka, India
PES1UG23AM203

Gayathri M

Department of Computer Science and Engineering
PES University
Bengaluru, Karnataka, India
PES1UG23CS217

Preetham Kumar S

Department of Computer Science and Engineering
PES University
Bengaluru, Karnataka, India
PES1UG23AM213

Abstract—Automated forecasting systems that train many candidate models must decide which one to serve, and a common design is to rank candidates by a weighted sum of several criteria—accuracy, forecast stability, and computational cost—on the reasoning that a single error metric is an incomplete basis for practical model selection. This paper subjects one such composite score to a held-out evaluation and reports that it does not work. We benchmark a six-term composite score, deployed in a nineteen-model retail forecasting system, against 487 series drawn from five aggregation levels of the M5 competition hierarchy, using a test window never exposed to model selection and reporting mean absolute scaled error. The composite score is outperformed by ranking on cross-validated RMSE alone at every aggregation level, decisively so where the comparison is adequately powered ($p = 2.1 \times 10^{-7}$ at store $\times$ department, $p = 3.6 \times 10^{-6}$ at item level). We trace the cause and find it structural rather than a poor choice of weights: each criterion is normalized by its own Tukey fence, which bounds the denominator but not the resulting ratio, so training time—spanning six orders of magnitude across the model registry—produces normalized values reaching 43 while RMSE, spanning a factor of 1.2, remains within [0.5, 1.0]. A term carrying a nominal weight of 0.10 consequently determines between 63% and 88% of all selections, and the models it excludes are precisely those that generalize best. Neither re-weighting nor a corrected normalization recovers the gap. Re-expressing the same cost requirement as a constraint rather than a scoring term does: selecting the most accurate model that trains within a fixed budget beats single-metric selection by approximately 5% MASE on fine-grained series (five of five folds, 95% confidence interval excluding zero, stable across random seeds) while consuming an order of magnitude less computation, and degenerates gracefully to unconstrained selection at coarse aggregation where no constraint is warranted. We further show that selecting a scoring configuration on the same data used to evaluate it inflates apparent performance by 0.12–0.20 MASE, exceeding every effect reported here, and

that nested cross-validation over series is therefore necessary rather than optional. We conclude that heterogeneous criteria differing by orders of magnitude in scale should not be combined additively, and that cost belongs in the constraint set rather than the objective.

Index Terms—demand forecasting, model selection, multi-criteria decision analysis, time series analysis, retail analytics, constrained optimization, cross-validation, M5 competition

I. Introduction

A. Background

Accurate demand forecasting underlies nearly every operational decision a retail organization makes: inventory replenishment, staffing, pricing, promotional planning, and supply-chain coordination all depend on a credible estimate of how much of a given product will sell over a future horizon [1]. Retail sales series are shaped simultaneously by long-run trend, weekly and annual seasonality, promotional events, price changes, and idiosyncratic noise, and different products within the same organization can exhibit qualitatively different demand patterns, from smooth, high-volume staples to sparse, intermittent demand for slow-moving or newly introduced items [2]. No single forecasting technique performs best across this diversity of patterns, a conclusion repeatedly reinforced by large-scale forecasting competitions such as the M4 and M5 competitions [3], [4].

B. Business Motivation

For small and mid-sized retailers in particular, the practical obstacle is rarely the absence of a suitable algorithm in the academic literature; it is the absence of

a system that (i) works across heterogeneous, often messy uploaded datasets without bespoke data-engineering effort, (ii) selects an appropriate model automatically rather than requiring in-house data-science expertise, and (iii) produces an output that a non-technical manager can trust and act upon. A forecast that is numerically accurate but delivered as an opaque number, with no explanation of why a particular model was chosen or how confident the system is, is of limited practical value to a retail operations team.

C. Challenges in Retail Forecasting

Three challenges recur throughout this problem space. First, heterogeneity: uploaded datasets vary in file format, column naming convention, date format, sampling frequency, and completeness, and a forecasting pipeline intended for general use must detect and normalize all of these automatically. Second, model selection under uncertainty: because no model dominates across all series, a credible system must train a diverse set of candidates and adjudicate among them using an evaluation procedure that is robust to a single badly-fit outlier model distorting the comparison. Third, intermittent and small-sample regimes: many retail items sell too infrequently, or a dataset is too short, for computationally expensive models such as deep neural networks to be trained reliably, and a well-designed system should recognize this rather than train models likely to overfit or fail silently.

D. Machine Learning and Deep Learning Approaches

Statistical time-series methods such as ARIMA and Holt-Winters exponential smoothing [5], [6] remain strong baselines, particularly on short series, and modern additive-regression tools such as Prophet [7] extend this family with automatic seasonality and holiday-effect modeling. Tree-based machine learning models, most notably random forests [8] and gradient-boosted trees [9], have proven competitive on tabular, feature-engineered representations of time series, while recurrent architectures such as LSTM and GRU networks [10], [11], convolutional sequence models such as temporal convolutional networks [12], and attention-based transformer architectures [13] have extended the state of the art on longer, richer series, at the cost of requiring substantially more training data to avoid overfitting [14].

E. The Need for Explainable AI

A parallel and increasingly important line of work addresses the interpretability of forecasting and machine learning systems more generally. Post-hoc explanation techniques such as SHAP [15] and LIME [16] have become common tools for exposing feature-level contributions to individual predictions, and the broader emergence of instruction-following large language models [17], [18] has opened a complementary avenue: rather than only exposing numerical attributions, a system can generate

a natural-language explanation of its own output. When such a language model is grounded in retrieved domain knowledge and in the system’s own deterministic computations, using a retrieval-augmented generation architecture [19], the risk of the explanation contradicting or fabricating facts about the underlying forecast can be substantially reduced. This is the approach adopted in the system described in this paper.

F. Research Motivation

The motivation for this work is therefore not to propose a new forecasting algorithm, but to design and implement a complete, end-to-end system that (i) is dataset-agnostic across retail, transactional, and general time-series data; (ii) trains and fairly adjudicates a broad ensemble of nineteen models spanning statistical, machine learning, deep learning, and intermittent-demand families; (iii) degrades gracefully rather than failing when a dataset is too small or a candidate model is unsuitable; and (iv) closes the interpretability gap using a combination of native feature-importance scores and a grounded, retrieval-augmented language model layer, rather than treating the forecast as an opaque endpoint.

G. Contributions

The contributions of this paper are summarized as follows.

- A unified orchestration layer that trains nineteen forecasting models (seven statistical, four classical machine learning, seven deep learning, and one intermittent-demand model) concurrently on an arbitrary uploaded time series, using an adaptive walk-forward cross-validation scheme whose fold count scales with the available data.
- A robust, multi-criteria model-selection score that normalizes root-mean-squared error, mean absolute error, mean absolute percentage error, forecast stability, computation time, and coefficient of determination using a Tukey-fence-based robust scale, preventing a single catastrophically mis-fit model from distorting the ranking of the remaining candidates.
- A frequency-aware gating mechanism that withholds deep learning models from datasets below an empirically justified minimum row count, communicating this decision explicitly to the end user rather than training and reporting on an unreliable model.
- Integration of real exogenous features (price and promotional indicators), when present in the uploaded dataset, into the gradient boosting, random forest, and seasonal autoregressive models, with an explicit and conservative policy for extending these features into the unobserved forecast horizon.
- A retrieval-augmented, large-language-model explanation layer, grounded simultaneously in a curated retail-domain knowledge base and in the pipeline’s

own deterministic metrics, that provides a conversational question-answering interface and a structured business insights report, with explicit prompt-level constraints preventing the language model from contradicting the system’s own computed labels.

- An empirical evaluation of the complete pipeline on a real retail transaction dataset, reporting accuracy, computation time, and the practical divergence between single-metric and multi-criteria model selection.

H. Paper Organization

The remainder of this paper is organized as follows. Section II reviews related work in retail and time-series forecasting, machine learning and deep learning approaches, and explainable artificial intelligence. Section III states the problem addressed and the corresponding research gap. Section IV describes the proposed system architecture and workflow. Section V describes the dataset used for evaluation and the automated preprocessing pipeline. Section VI details every implemented forecasting model, including governing equations. Section VII describes the explainability layer. Section VIII presents the system architecture diagrams. Section IX describes the experimental setup. Section X reports quantitative results. Section XI interprets these results. Section XII outlines future work, and Section XIII concludes the paper.

II. Literature Review

A. Retail and Demand Forecasting

Retail forecasting has a long history as an applied field distinct from generic time-series analysis, owing to domain-specific complications such as promotions, price elasticity, new-product introduction, and stockouts [1], [20]. Large-scale empirical competitions have played a disproportionate role in establishing what actually works in practice. The M4 competition [3] compared 61 forecasting methods across 100,000 series and found that combination methods and, later, hybrid statistical-machine-learning approaches outperformed both pure statistical and pure machine learning methods in isolation. The subsequent M5 competition [4] specifically targeted retail sales data from a large retailer and confirmed that gradient-boosted tree methods were highly competitive on real retail series, while also highlighting the difficulty of forecasting intermittent, low-volume items accurately. These findings directly motivate the multi-model, automatically-adjudicated design adopted in this paper, rather than committing to a single model family a priori.

B. Statistical Time-Series Forecasting

The Box-Jenkins ARIMA methodology [5] and its seasonal extension SARIMA remain foundational statistical approaches, modeling a series as a function of its own past values and past forecast errors after appropriate differencing. Exponential smoothing methods,

including the Holt-Winters method [6], model level, trend, and seasonal components multiplicatively or additively and remain highly competitive baselines, particularly for shorter series where more complex models are prone to overfitting [21], [22]. More recently, additive regression frameworks such as Prophet [7] were designed explicitly for business forecasting at scale, decomposing a series into trend, multiple seasonalities, and holiday effects, and are notable for degrading gracefully on noisy or irregular business data.

C. Machine Learning for Time Series

Framing time-series forecasting as a supervised regression problem over engineered lag, rolling-statistic, and calendar features allows conventional machine learning regressors to be applied directly. Random forests [8] and gradient-boosted trees, particularly the XGBoost implementation [9], have become standard strong baselines in this framing and were prominent among top-performing methods in the M5 competition [4]. Instance-based methods such as k -nearest-neighbor regression [23] and kernel methods such as support vector regression [24] are comparatively lightweight alternatives that can be competitive on smaller datasets where tree ensembles or neural networks have insufficient data to realize their capacity advantage.

D. Deep Learning for Time Series

Recurrent neural architectures, especially the long short-term memory (LSTM) network [10], [25] and the gated recurrent unit (GRU) [11], were designed to model long-range temporal dependence while mitigating the vanishing-gradient problem of simple recurrent networks. Convolutional alternatives, including the temporal convolutional network [12], achieve comparable or superior performance on some sequence-modeling benchmarks with better parallelization. Attention-based transformer architectures [13] have subsequently been adapted specifically for long-sequence time-series forecasting, for example in the Informer architecture [26], while specialized forecasting-native deep architectures such as DeepAR [27] and N-BEATS [28] target probabilistic and interpretable forecasting respectively. A consistent finding across this literature, and a central empirical justification for the frequency-aware gating mechanism introduced in this paper, is that deep sequence models require substantially more training observations than statistical or tree-based methods to generalize reliably [14]; when applied to short series they are prone to memorizing noise rather than learning genuine temporal structure.

E. Hybrid and Meta-Learning Approaches

A separate line of work combines statistical and machine-learning components directly, for example residual-correction hybrids in which a statistical model captures linear trend and seasonality while a neural

network models the remaining nonlinear residual [29]. Meta-learning approaches for retail forecasting [30] instead learn to select or weight among candidate models based on series characteristics. The system described in this paper adopts a related but simpler and more transparent strategy: every candidate model is always trained and evaluated identically via walk-forward cross-validation, and selection is performed by an explicit, auditable composite scoring rule rather than a learned meta-model, prioritizing transparency over marginal accuracy gains.

F. Intermittent Demand Forecasting

Standard forecasting methods perform poorly on sparse, intermittent demand series in which a large fraction of periods have zero recorded demand, a pattern common for slow-moving retail items and spare parts. Croston’s method [2] addresses this by separately smoothing the non-zero demand size and the inter-demand interval and forecasting their ratio as a constant rate. The Syntetos-Boylan approximation [31] subsequently identified and corrected a systematic positive bias in Croston’s original estimator, and is the variant adopted in this paper’s implementation.

G. Evaluation Methodology for Time Series

Correctly evaluating time-series models requires respecting temporal ordering; naive random train-test splits leak future information into the training set and inflate reported accuracy. Walk-forward, expanding-window cross-validation is the accepted alternative [32], and the number of folds used should reasonably reflect the amount of data available, since too few folds increases the risk that a model appears best due to chance performance on a single split [33]. Broader reviews of forecasting practice [34] additionally emphasize that a single point accuracy metric such as root-mean-squared error is an incomplete criterion for model selection in practice, since it does not capture computational cost, forecast stability, or the risk profile of a badly-calibrated model; this observation directly motivates the multi-criteria composite scoring approach described in Section VI.

H. Explainable AI and Retrieval-Augmented Generation

Post-hoc, model-agnostic explanation techniques such as SHAP [15], grounded in cooperative game-theoretic Shapley values, and LIME [16], based on local surrogate modeling, are widely used to attribute a model’s prediction to its input features. Tree-based models additionally expose a native, computationally inexpensive impurity- or gain-based feature-importance measure as a byproduct of training, which the system described in this paper surfaces directly for its gradient boosting and random forest models rather than computing a separate post-hoc attribution.

Independently, the emergence of large, instruction-following language models [17], [18] has introduced natural-language generation as a complementary explanation modality: rather than exposing numerical attributions

alone, a system can narrate its reasoning in prose intelligible to a non-technical stakeholder. The central risk of this approach is hallucination, the generation of fluent but factually incorrect statements. Retrieval-augmented generation [19] mitigates this risk by conditioning language-model generation on retrieved, verifiable source documents rather than relying purely on the model’s parametric knowledge. The explanation layer described in this paper applies this principle twice: qualitative business advice is grounded in a retrieved, curated retail-domain knowledge base, while all quantitative claims are grounded directly in the deterministic metrics computed by the forecasting pipeline itself, with explicit prompt-level instructions preventing the language model from re-deriving or contradicting labels the system has already computed.

I. Positioning of This Work

Relative to the reviewed literature, this paper does not propose a novel forecasting algorithm. Its contribution is architectural and systems-oriented: it integrates a broad, heterogeneous ensemble spanning all four model families discussed above behind a single automated pipeline, introduces an explicit and auditable robust multi-criteria selection rule in place of raw error-metric ranking, applies an empirically justified frequency-based gate to prevent unreliable deep-learning training on insufficient data, and couples the resulting forecasts to a retrieval-augmented natural-language explanation layer that is explicitly constrained to remain consistent with the system’s own deterministic outputs.

III. Problem Statement

A. Current Limitations

Existing retail forecasting tooling accessible to non-specialist organizations typically exhibits one or more of the following limitations. First, many tools commit to a single model family, such as a single statistical method or a single machine learning regressor, and therefore perform inconsistently across the heterogeneous demand patterns present even within one retailer’s catalogue, a pattern well documented by cross-method forecasting competitions [3], [4]. Second, tools that do support multiple models frequently select among them using only a single point-accuracy metric, most commonly root-mean-squared error, which is sensitive to the scale of the series and does not account for computation cost, forecast stability, or the risk that one poorly calibrated model can distort a comparison against otherwise well-fitted competitors. Third, deep learning models are often applied uniformly regardless of dataset size, despite established evidence that they require substantially more observations than statistical or tree-based alternatives to generalize reliably [14]; on short retail series this risks reporting an overfit model as though it were reliable. Fourth, and most consequential for adoption by non-technical stakeholders, the forecast is typically delivered as a bare number or chart with no

accompanying explanation of model choice, uncertainty, or business implication, which limits the degree to which a retail manager can trust and act on the output.

B. Research Gap

The literature reviewed in Section II addresses these limitations largely in isolation: forecasting-competition research establishes which model families perform well on which data characteristics, evaluation-methodology research establishes how models should be validated, and explainable-AI research establishes how a trained model’s behavior can be attributed or narrated. There is comparatively little published work describing a single, integrated system that combines a broad multi-family model ensemble, a robust and auditable multi-criteria selection rule, an explicit data-driven gate on which models are even attempted, and a language-model explanation layer that is architecturally constrained to remain factually consistent with the system’s own deterministic computations. This integration gap, rather than any single algorithmic deficiency, is the specific research gap this paper addresses.

C. Objectives

The system described in this paper is designed to satisfy the following objectives.

- 1) Accept an arbitrary time-indexed dataset in common tabular file formats and automatically identify the date column, the numeric target column, and any usable exogenous or grouping columns, without requiring the uploader to pre-process the file.
- 2) Train a broad ensemble of forecasting models spanning statistical, classical machine learning, deep learning, and intermittent-demand families, evaluated identically via walk-forward cross-validation.
- 3) Select the best-performing model using a composite criterion that is robust to a single outlier model and that reflects accuracy, stability, and computational cost jointly, rather than a single error metric alone.
- 4) Withhold computationally expensive deep learning models from datasets below a data-driven minimum size, and communicate this decision transparently rather than silently degrading forecast quality.
- 5) Incorporate real exogenous business features, specifically price and promotional indicators, into the subset of models capable of using them, when such columns are present in the uploaded dataset.
- 6) Provide a natural-language explanation layer, grounded in retrieved domain knowledge and in the pipeline’s own deterministic outputs, that answers user questions and produces a proactive business report without contradicting the system’s own computed metrics or labels.

D. Expected Improvements

Relative to a single-model or single-metric baseline, the proposed system is expected to improve robustness across

heterogeneous retail series by virtue of its broad ensemble and adaptive model gating, to improve the trustworthiness of reported uncertainty by deriving forecast intervals from each model’s own historical error rather than an arbitrary fixed percentage, and to improve the practical usability of the forecast for non-technical stakeholders through grounded natural-language explanation. These improvements are evaluated empirically in Sections IX and X.

IV. Proposed System

A. System Overview

The proposed system is a full-stack web application comprising a Python FastAPI backend and a React single-page frontend, communicating over a REST HTTP API. The backend is organized into four logical layers: (i) an ingestion and preprocessing layer that accepts an uploaded file and produces a clean, gap-filled, outlier-capped time series together with any usable exogenous features; (ii) a model orchestration layer that trains the full ensemble of candidate forecasting models concurrently and computes a composite selection score; (iii) a persistence layer that caches forecast results and conversational state to disk, keyed by a client-generated session identifier, so that concurrent users or browser tabs do not overwrite one another’s results and so that state survives a backend restart; and (iv) an explanation layer that generates natural-language narration of the forecast, grounded jointly in retrieved domain knowledge and in the pipeline’s own deterministic metrics. Figure 1 depicts the overall architecture.

B. Workflow Overview

From the end user’s perspective, the workflow proceeds in four steps: (1) a dataset file is uploaded and the system responds with detected columns and suggested date, target, grouping, and exogenous-feature columns; (2) the user confirms or adjusts these column selections and a forecast horizon, and optionally selects a country code for holiday-effect modeling or a grouping value to forecast an individual product, store, or category; (3) the backend executes the full training and selection pipeline and returns the leaderboard of all attempted models together with the selected best model’s forecast, uncertainty interval, and, where applicable, feature importance; and (4) the user may inspect a model-comparison chart, ask the conversational assistant questions about the result, or request a structured business insights report.

C. Data Pipeline

The ingestion layer accepts CSV, Microsoft Excel (.xlsx/.xlsm), and JSON files. Column detection proceeds by a combination of dtype inspection, column-name keyword matching against curated vocabularies for date-like, target-like, grouping-like, price-like, and promotion-like columns, and, for date columns specifically, an ordered

cascade of parsing strategies including native pandas inference, explicit format cycling, Unix-timestamp interpretation bounded to a plausible calendar-year range, and finally mixed-format parsing as a last resort. Once a date and target column are selected, the preprocessing pipeline (i) parses and coerces the target column to a numeric type, handling currency symbols, thousands separators, and both American and European decimal conventions; (ii) sets the date column as index and aggregates duplicate timestamps by summation, consistent with the assumption that duplicate timestamps represent multiple transactions on the same period; (iii) infers the dominant sampling frequency and reindexes to a complete, gap-free calendar at that frequency; (iv) fills newly introduced and pre-existing missing values through time-aware interpolation followed by forward- and backward-filling; (v) detects and Winsorizes outliers using an interquartile-range rule; and (vi) applies light rolling-median smoothing for high-frequency (sub-daily) series. If exogenous feature columns were selected, they are aligned to the same cleaned index using an aggregation rule appropriate to their type, price-like columns by mean and promotional-flag columns by maximum, and the resulting exogenous frame is retained separately from, but index-aligned with, the cleaned target series.

D. Training Pipeline

The orchestration layer submits every registered model to a shared thread pool and trains them concurrently, subject to a per-model timeout appropriate to its statistical, machine-learning, or deep-learning category. Each model is evaluated using walk-forward, expanding-window cross-validation, with the number of folds determined adaptively from the number of available observations, subject to a configured minimum and maximum. Deep learning models are withheld entirely, rather than trained and reported unreliably, when the series has fewer observations than an empirically justified minimum for its inferred sampling frequency, and this decision is surfaced explicitly to the end user together with its justification. When exogenous features are present, a request-scoped override of the model registry substitutes exogenous-aware variants of the gradient boosting, random forest, and seasonal autoregressive models, leaving the remaining models untouched, so that the shared orchestration logic itself does not need to be aware of which individual models accept additional arguments.

E. Prediction Pipeline and Model Selection

Once every model has completed training and cross-validation, or timed out or raised an exception, the selection layer first filters to models that both completed successfully and produced a numerically usable forecast, defined as a forecast that is entirely finite and does not deviate implausibly from the historical distribution of the series. Among the surviving candidates, a composite

score is computed from six components, normalized root-mean-squared error, mean absolute error, mean absolute percentage error, a discontinuity-and-volatility stability penalty, a computation-time penalty, and the coefficient of determination, described formally in Section VI. The model with the lowest composite score is selected, and its forecast, together with an uncertainty interval derived from its own historical cross-validation error, is returned as the pipeline’s headline result, while every model’s individual leaderboard entry is retained for inspection.

F. Explainability Pipeline

The explanation layer operates on the deterministic output of the prediction pipeline; it never has access to, and cannot alter, the underlying model training or selection. Two complementary mechanisms are provided. First, for the gradient boosting and random forest models, the native impurity- or gain-based feature-importance scores produced during training are exposed directly. Second, a retrieval-augmented large-language-model layer accepts a natural-language question or generates a proactive report, retrieves the most relevant passages from a curated retail-domain knowledge base using embedding similarity, and combines this retrieved context with a deterministic textual summary of the forecast’s own metrics, trend classification, and, where applicable, exogenous feature usage, before generating a response. The prompt supplied to the language model explicitly enumerates the system’s own computed labels and instructs the model never to re-derive or contradict them.

G. User Interaction and Frontend Workflow

The frontend is implemented as a single-page React application. It presents a sequential workflow, upload, column configuration, and results, and, once results are available, exposes tabbed views for the historical-and-forecast chart, a log-scale model-comparison chart across the full leaderboard, a forecast confidence-band chart, and, when the selected model exposes it, a feature-importance chart. A conversational assistant panel and a dedicated business insights report page are reachable from the results view. All charts are constructed from data returned directly by the backend rather than fabricated client-side, including the historical series itself and the uncertainty interval.

H. API Communication

The frontend and backend communicate exclusively through a versioned REST API implemented with FastAPI, using JSON request and response bodies. The principal endpoints are an upload endpoint that returns detected columns and suggestions; a forecast endpoint that accepts a filename, date and target column selection, forecast horizon, and optional exogenous-feature, grouping, country, and multi-target parameters, and returns the full leaderboard and selected model; a cached-result

retrieval endpoint keyed by session identifier; a per-group ranking endpoint; a conversational endpoint; and a report-generation endpoint. Cross-origin resource sharing is restricted to an explicit allow-list of development origins, extensible via an environment variable for a deployed frontend origin.

I. Storage and Persistence

The system deliberately does not employ a relational or document database. Uploaded files are persisted to a dedicated directory on disk, referenced thereafter by a randomly generated filename to avoid collisions. Preprocessed series are separately persisted, both as a serialized object for programmatic reuse and as a plain comma-separated file for inspection. The retrieval-augmented explanation layer maintains a local, persistent vector store for the retail-domain knowledge-base embeddings. Finally, session-scoped state, namely the most recent forecast result and any conversational history, is held in an in-memory dictionary for low-latency access during normal operation and is additionally mirrored to a per-session file on disk, so that a backend process restart does not discard active sessions; this file-based mirror was judged sufficient for a single-process deployment and avoids introducing an external database dependency that the current deployment scope does not otherwise require.

V. Dataset Description

A. Primary Benchmark: M5

All comparative results in Section X use the M5 competition dataset [4], comprising daily unit sales of 3,049 products across ten Walmart stores in three US states over 1,941 days, together with per-product weekly selling prices and a calendar of events and SNAP benefit days. M5 was selected for three properties this study requires. Its series are long enough that all nineteen models, including the deep learning family gated below 500 observations, train on every series. Its twelve-level aggregation hierarchy allows the same comparison to be repeated at systematically different granularities, turning aggregation level into a controlled experimental variable rather than an uncontrolled property of a single dataset. And it supplies enough independent series for the statistical comparisons of Section IX to have power.

Series were extracted at levels 3, 5, 9, 10, and 12 of the standard M5 hierarchy (Table VI). Sales were aggregated by summation over the grouping keys; prices were averaged, unweighted, over the constituent item-store pairs. At item-level granularities, observations preceding a product’s first recorded sale were removed, since in M5 a leading run of zeros denotes a product not yet stocked rather than an absence of demand, and series with fewer than 30% non-zero days were excluded. M5 provides no explicit promotion indicator; a markdown proxy was derived by flagging days on which the selling price falls at least 5% below a trailing eight-week maximum. This

TABLE I: Raw Dataset Schema

Column	Type	Role
Transaction ID	Integer	Identifier (excluded)
Date	String (date)	Time index
Customer ID	String	Identifier (excluded)
Gender	Categorical	Not used in target series
Age	Integer	Not used in target series
Product Category	Categorical	Optional grouping dimension
Quantity	Integer	Alternative target candidate
Price per Unit	Integer	Optional exogenous (price)
Total Amount	Integer	Target variable (used)

proxy fires on 21% of series at level 9 and 45% at level 12, a limitation discussed in Section XI.

B. Secondary Dataset: Transaction Log

A second dataset is retained to characterize system behavior in a low-signal regime. It is a retail transaction log consisting of 1,000 individual sale transactions recorded between January 1, 2023 and January 1, 2024. Each row represents a single transaction and contains nine columns: a transaction identifier, a transaction date, a customer identifier, customer gender, customer age, a product category (Beauty, Clothing, or Electronics), the quantity of items purchased, the price per unit, and the total transaction amount. Table I summarizes the raw schema. The dataset contains no missing values at the transaction level; all data-quality handling described below arises from aggregating irregular transaction timestamps into a regular daily calendar, not from missing source fields.

C. Target Variable

The target variable forecast in this evaluation is Total Amount, aggregated by summation across all transactions sharing the same calendar date, yielding a single daily total-sales series. This aggregation is performed automatically by the preprocessing pipeline described in Section IV-C rather than by manual data preparation, since duplicate timestamps are, by design, summed under the assumption that they represent multiple transactions within the same period.

D. Automated Column Detection

Presented with the raw file, the upload endpoint automatically identified Date as the date column and flagged Total Amount, Quantity, and Price per Unit as numeric columns with plausible target semantics based on column-name keyword matching, correctly suggesting Date as the date column and, in this evaluation run, Quantity was the column ranked first by the keyword heuristic; Total Amount was selected manually for this evaluation as the

TABLE II: Preprocessing Steps Applied (Logged at Run Time)

#	Transformation
1	Date column parsed to a datetime index.
2	Target column coerced to a numeric (float) type.
3	Index sorted chronologically (2023-01-01 to 2024-01-01).
4	655 duplicate timestamps aggregated by summation (1,000 $\rightarrow$ 345 rows).
5	Calendar gaps filled; 21 missing dates added (366 rows total).
6	21 missing values filled by time-interpolation, forward-, and backward-fill.
7	Outlier capping (IQR $\times$ 2.5): 3 values capped to an upper bound of 6,041.88.
8	High-frequency smoothing skipped (not applicable at daily frequency).

TABLE III: Descriptive Statistics of the Cleaned Daily Series

Statistic	Value
Observations (days)	366
Minimum	25.00
Maximum (post-capping)	6,041.88
Mean	1,299.89
Median	1,060.00
Standard deviation	1,148.30
Inferred frequency	Daily

more business-relevant revenue target. Product Category was correctly identified as a candidate grouping dimension.

E. Preprocessing and Cleaning

Table II reports the exact transformations applied by the automated preprocessing pipeline for this dataset, exactly as logged by the system at run time. The raw 1,000 transactions, spanning 345 distinct calendar dates, were aggregated by date-wise summation, after which 655 duplicate-timestamp rows were consolidated. The resulting irregular 345-point series was then reindexed to a complete daily calendar spanning January 1, 2023 to January 1, 2024 (366 calendar days inclusive), introducing 21 previously absent calendar dates with no recorded transactions. The values at these newly introduced dates, together with no other missing values, were filled by time-aware linear interpolation. Finally, interquartile-range-based outlier capping (multiplier 2.5) identified three days with anomalously high aggregated sales and capped them to an upper bound of 6,041.88, leaving a final clean series of 366 daily observations with mean 1,299.89, standard deviation 1,148.30, and median 1,060.00, summarized in Table III.

F. Feature Engineering

For the machine learning and deep learning model families described in Section VI, the cleaned univariate series is transformed into a supervised-learning representation consisting of lagged values at eight lags (1, 2, 3, 7, 14, 21, 28, and 30 days), rolling mean, standard deviation,

minimum, and maximum statistics over 7-, 14-, and 30-day windows, exponentially weighted moving averages at two spans, and calendar features including day-of-week, month, quarter, day-of-year, week-of-year, month-boundary indicators, and sine-cosine cyclical encodings of day-of-week and month. This feature representation is identical across the gradient boosting, random forest, and instance-based models to ensure a fair comparison. Where price and promotional columns are present and selected, as described in Section IV-C, they are appended to this feature matrix directly.

G. Normalization

Feature matrices supplied to the gradient boosting, random forest, k -nearest-neighbor, and support-vector models are scaled using a robust scaler based on the median and interquartile range, rather than a standard mean-variance scaler, since raw retail sales series frequently contain heavy-tailed spikes even after outlier capping. Deep learning models instead use min-max scaling to the unit interval, fit on the training partition only and applied unchanged to validation and forecast data, consistent with standard practice for recurrent and convolutional sequence models.

H. Train-Validation Partitioning

No single fixed train-test split is used. Instead, every model is evaluated using walk-forward, expanding-window cross-validation (Section VI), with the number of folds determined adaptively from the series length. For this 366-observation dataset, the adaptive rule yields three cross-validation folds per model, after which every model is refit on the complete cleaned series for the final reported forecast.

I. Dataset Scale Relative to Model Requirements

At 366 daily observations, this dataset falls below the deep-learning row-count threshold described in Section VI, and the seven deep learning models were consequently withheld automatically for this evaluation run, a decision reported explicitly by the system rather than silently applied. The remaining twelve statistical, machine learning, and intermittent-demand models were all trained and evaluated successfully, and the corresponding results are reported in Section X.

VI. Methodology

This section describes every one of the nineteen forecasting models implemented in the system, organized by family, together with the shared cross-validation procedure, evaluation metrics, and the composite model-selection score.

A. Statistical Models

1) **ARIMA**: The autoregressive integrated moving average model [5] models a differenced series as a linear function of its own past values and past error terms. For a series y_t differenced d times to obtain stationarity, an ARIMA(p, d, q) model is

$$\left(1 - \sum_{i=1}^p \phi_i L^i\right) (1 - L)^d y_t = \left(1 + \sum_{j=1}^q \theta_j L^j\right) \varepsilon_t, \quad (1)$$

where L is the lag operator, ϕ_i and θ_j are the autoregressive and moving-average coefficients, and ε_t is white noise. The implementation performs a grid search over $p, q \in \{0, 1, 2\}$ (bounded to keep search cost tractable) selecting the order minimizing Akaike Information Criterion on an initial training partition, and reuses this order across all cross-validation folds and the final refit.

2) **SARIMA** and **SARIMAX**: The seasonal extension augments Eq. (1) with seasonal autoregressive and moving-average polynomials of order $(P, D, Q)_m$ at seasonal period m , inferred automatically from the detected sampling frequency (for example $m = 7$ for daily data). SARIMAX additionally admits an exogenous regression term $\beta^\top \mathbf{x}_t$. In the implementation, the exogenous vector $\mathbf{x}_t$ always includes calendar cyclical features (sine-cosine encodings of month and day-of-week, and a month-end indicator) and, when available and selected by the user, real price and promotional features. To bound computational cost, the order search and per-fold refitting are simplified for seasonal periods exceeding a configured threshold, since state-space estimation cost scales approximately with m^2 .

3) **Holt-Winters Exponential Smoothing**: The Holt-Winters method [6] recursively updates level ℓ_t , trend b_t , and seasonal s_t components. In additive form,

$$\ell_t = \alpha(y_t - s_{t-m}) + (1 - \alpha)(\ell_{t-1} + b_{t-1}), \quad (2)$$

$$b_t = \beta(\ell_t - \ell_{t-1}) + (1 - \beta)b_{t-1}, \quad (3)$$

$$s_t = \gamma(y_t - \ell_t) + (1 - \gamma)s_{t-m}, \quad (4)$$

$$\hat{y}_{t+h} = \ell_t + h b_t + s_{t+h-m[h/m]}, \quad (5)$$

with an analogous multiplicative form used when the series exhibits variance that scales with its level. The implementation grid-searches additive and multiplicative trend and seasonal combinations under cross-validation, with damped-trend smoothing enabled throughout, and reduces to plain (non-seasonal) exponential smoothing when fewer than two full seasonal cycles are available.

4) **Moving Average**: A simple moving-average baseline forecasts the next value as the mean of the preceding w observations,

$$\hat{y}_{t+1} = \frac{1}{w} \sum_{i=0}^{w-1} y_{t-i}, \quad (6)$$

with the window length w selected by cross-validation from a candidate set, and multi-step forecasts generated recursively.

5) **Prophet**: Prophet [7] models a series as an additive (or, when the coefficient of variation of the series suggests multiplicative seasonality, multiplicative) decomposition

$$y_t = g(t) + s(t) + h(t) + \varepsilon_t, \quad (7)$$

where $g(t)$ is a piecewise-linear or logistic trend with automatically detected changepoints, $s(t)$ represents Fourier-series yearly and weekly seasonal terms, and $h(t)$ represents holiday effects. The implementation enables yearly and weekly seasonality conditionally on series length, sets the changepoint prior scale to 0.1 and the changepoint range to 0.9, and optionally incorporates a specific country's public holidays into $h(t)$ when a country code is supplied by the user, via the model's native country-holiday mechanism.

B. Machine Learning Models

All four machine learning models in this subsection are trained on the identical engineered feature representation described in Section V (lags, rolling statistics, exponentially weighted moving averages, and calendar features), constructed from the detrended residual of the series rather than the raw series itself. A global linear trend

$$\hat{g}(t) = a + bt \quad (8)$$

is fit by ordinary least squares over the integer time index t , and each tree-based or instance-based model is trained to predict the residual $r_t = y_t - \hat{g}(t)$. This detrending step is necessary because tree-based and instance-based regressors cannot extrapolate outside the range of target values observed during training and would otherwise plateau on a trending series; predicting the stationary residual and adding the extrapolated trend back at forecast time avoids this failure mode.

1) **XGBoost**: Gradient-boosted trees [9] construct an additive ensemble $F(\mathbf{x}) = \sum_{k=1}^K f_k(\mathbf{x})$ of regression trees, minimizing the regularized objective

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2, \quad (9)$$

where ℓ is the squared-error loss, T is the number of leaves of tree f , w are its leaf weights, and γ, λ penalize tree complexity and leaf-weight magnitude respectively. The implementation trains with 500 boosting rounds under early stopping (patience of 30 rounds evaluated on the held-out cross-validation fold), a learning rate of 0.05, a maximum tree depth of 5, subsample and column-subsample fractions of 0.8, L_1 regularization (α) of 0.1, and L_2 regularization (λ) of 1.0; the final model is refit on the complete series using the number of boosting rounds selected by early stopping during cross-validation. Gain-based feature importances are retained from the final fit for the explainability layer (Section VII).

2) Random Forest: The random forest [8] instead constructs an ensemble of K regression trees, each trained on a bootstrap resample of the training data with a random subset of features considered at each split, and averages their predictions,

$$\hat{y} = \frac{1}{K} \sum_{k=1}^K T_k(\mathbf{x}). \quad (10)$$

The implementation uses 100 trees, a minimum split size of 5, a minimum leaf size of 2, and considers $\sqrt{p}$ features at each split, where p is the total feature count. Out-of-bag R^2 is retained as a free internal validation estimate, and a 95% prediction interval is derived from the empirical standard deviation across individual trees' predictions at each forecast step, in place of the generic cross-validation-error-based interval used for models without a native uncertainty estimate.

3) k -Nearest-Neighbor Regression: The k -nearest-neighbor regressor [23] predicts the (distance-weighted) average target value of the k nearest training points in feature space,

$$\hat{y}(\mathbf{x}) = \frac{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i y_i}{\sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i}, \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)}, \quad (11)$$

using Euclidean distance $d(\cdot, \cdot)$ on the robust-scaled feature representation. The implementation cross-validates $k \in \{3, 5, 7, 10, 15\}$, subject to k remaining below half the available fold size.

4) Support Vector Regression: Support vector regression [24] fits a function that deviates from each training target by at most ϵ , while remaining as flat as possible, formalized as

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \quad (12)$$

subject to $y_i - (\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \leq \epsilon + \xi_i$ and its symmetric counterpart, with slack variables $\xi_i, \xi_i^* \geq 0$ and a radial basis function kernel ϕ . The implementation cross-validates the penalty parameter C and the tolerance ϵ , and scales the target itself with a robust scaler in addition to the feature matrix.

C. Deep Learning Models

All seven deep learning models share a common data representation, a sliding window of the preceding $L = 30$ time steps predicting the next single step, a common optimization configuration (Adam optimizer, learning rate 10^{-3} , batch size 32, weight decay 10^{-5} , plateau-based learning-rate reduction), and common regularization (dropout rate 0.2, gradient-norm clipping at 1.0, early stopping with a patience of 5 epochs over a maximum of 30 epochs), summarized alongside the XGBoost and Random Forest hyperparameters in Table IV. Every model

TABLE IV: Fixed Hyperparameters for the Gradient Boosting, Random Forest, and Deep Learning Models

Model	Hyperparameter	Value
XGBoost	Boosting rounds	500 (early stop)
	Learning rate	0.05
	Max tree depth	5
	Subsample fraction	0.8
	L_1 / L_2 reg. (α, λ)	0.1 / 1.0
	Early-stop patience	30 rounds
Random Forest	Number of trees	100
	Min. samples/split	5
	Min. samples/leaf	2
	Max. features/split	$\sqrt{p}$
Deep Learning (shared, all 7 architectures)	Lookback window	30 steps
	Hidden size	64
	Recurrent layers	2
	Dropout rate	0.2
	Optimizer	Adam
	Learning rate	10^{-3}
	Batch size / max epochs	32 / 30

minimizes the mean squared error between the predicted and true next-step value in min-max-scaled space,

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2. \quad (13)$$

Multi-step forecasts are produced recursively, feeding each predicted value back as the most recent input for the subsequent step. The implementation uses PyTorch [35] when available, with an automatic fallback to a Keras-based implementation of the same seven architectures otherwise.

1) RNN: The simple recurrent network updates a hidden state $\mathbf{h}_t = \tanh(W_h \mathbf{h}_{t-1} + W_x x_t + \mathbf{b})$ at each step and predicts from the final hidden state through a linear output layer, using a hidden size of 64 across 2 stacked layers.

2) LSTM: The long short-term memory network [10], [25] augments the simple recurrent update with input, forget, and output gates and a separate cell state $\mathbf{c}_t$,

$$\mathbf{f}_t = \sigma(W_f[\mathbf{h}_{t-1}, x_t] + \mathbf{b}_f), \quad (14)$$

$$\mathbf{i}_t = \sigma(W_i[\mathbf{h}_{t-1}, x_t] + \mathbf{b}_i), \quad (15)$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c[\mathbf{h}_{t-1}, x_t] + \mathbf{b}_c), \quad (16)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (17)$$

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, x_t] + \mathbf{b}_o), \quad (18)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \quad (19)$$

mitigating the vanishing-gradient problem of the simple recurrent update, again with hidden size 64 across 2 layers.

3) GRU: The gated recurrent unit [11] simplifies the LSTM gating to an update gate $\mathbf{z}_t$ and reset gate $\mathbf{r}_t$,

$$\mathbf{z}_t = \sigma(W_z[\mathbf{h}_{t-1}, x_t]), \quad (20)$$

$$\mathbf{r}_t = \sigma(W_r[\mathbf{h}_{t-1}, x_t]), \quad (21)$$

$$\tilde{\mathbf{h}}_t = \tanh(W[\mathbf{r}_t \odot \mathbf{h}_{t-1}, x_t]), \quad (22)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t, \quad (23)$$

with the same hidden size and depth as the LSTM variant.

4) BiLSTM: The bidirectional LSTM processes the input window in both forward and backward temporal order using two independent LSTM layers as in Eq. (19), concatenating their final hidden states before the output layer, allowing the representation at each step to incorporate both preceding and following context within the lookback window.

5) CNN-1D: The one-dimensional convolutional model applies a stack of temporal convolutions with kernel size 3 and ReLU activation directly over the input window, followed by a dense output layer, extracting local temporal patterns without recurrence.

6) Temporal Convolutional Network: The temporal convolutional network [12] stacks causal, dilated one-dimensional convolutions so that the receptive field grows exponentially with depth while preserving strict temporal causality (no leakage of future information), with residual connections between blocks.

7) Transformer: The transformer encoder [13] replaces recurrence with multi-head self-attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \quad (24)$$

computed over positionally encoded representations of the input window, allowing every time step to attend directly to every other time step within the lookback window regardless of distance, in contrast to the sequential propagation required by recurrent architectures.

D. Intermittent Demand: Croston's Method with the Syntetos-Boylan Correction

For series with a substantial fraction of zero-demand periods, the preceding models are known to perform poorly relative to methods designed specifically for sparse demand. Croston's method [2] separately applies simple exponential smoothing, with smoothing parameter α , to the non-zero demand sizes z and to the intervals q between successive non-zero periods,

$$\hat{z}_t = \alpha z_t + (1 - \alpha)\hat{z}_{t-1}, \quad (25)$$

$$\hat{q}_t = \alpha q_t + (1 - \alpha)\hat{q}_{t-1}, \quad (26)$$

and forecasts a constant future demand rate $\hat{y} = \hat{z}_t / \hat{q}_t$. The Syntetos-Boylan approximation [31] corrects a known positive bias in this estimator,

$$\hat{y}_{\text{SBA}} = \left(1 - \frac{\alpha}{2}\right) \frac{\hat{z}_t}{\hat{q}_t}, \quad (27)$$

and is the variant used in the implementation, with $\alpha = 0.1$. Because the forecast is a constant rate rather than a per-period curve, the model naturally competes on an equal footing with every other candidate under the shared cross-validation and selection procedure, and is expected to rank favorably specifically on sparse series and unfavorably on dense, trending, or seasonal series.

E. Cross-Validation Procedure

Every model is evaluated using walk-forward, expanding-window cross-validation [32] rather than a single fixed train-test split, respecting temporal ordering so that no fold is trained on data that temporally follows its validation window. The number of folds k is determined adaptively from the number of observations n ,

$$k = \min(k_{\max}, \max(k_{\min}, \lfloor n/100 \rfloor)), \quad (28)$$

with $k_{\min} = 2$ and $k_{\max} = 5$ in the implementation, so that datasets with more observations are evaluated with more folds, up to a bound that keeps the total re-fitting cost tractable for computationally expensive models.

F. Evaluation Metrics

Four standard regression metrics are computed for every fold and averaged across folds. With y_i the true value, $\hat{y}_i$ the predicted value, and n the number of evaluated points,

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}, \quad (29)$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad (30)$$

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|, \quad (31)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}. \quad (32)$$

G. Composite Multi-Criteria Selection Score

Rather than ranking candidate models by a single metric, most commonly RMSE, the implementation computes a composite score S for every successfully trained model as a weighted combination of six normalized components,

$$S = 0.40 \widetilde{\text{RMSE}} + 0.20 \widetilde{\text{MAE}} + 0.20 \widetilde{\text{MAPE}} + 0.10 \widetilde{\text{Stab}} + 0.10 \widetilde{\text{Speed}} - 0.10 \widetilde{\text{Bias}}, \quad (33)$$

where $\widetilde{(\cdot)}$ denotes normalization by a robust scale factor, and Speed is the mean of normalized training and prediction time. Each normalizing denominator is computed not as the raw maximum across candidate models, but as a Tukey outlier fence,

$$\text{fence} = Q_3 + 1.5(Q_3 - Q_1), \quad (34)$$

over the corresponding metric values across all successfully trained candidates, where Q_1 and Q_3 are the first and third quartiles. This choice is deliberate: normalizing by the raw

maximum allows a single catastrophically mis-fit model to compress every other model’s normalized error to near zero, at which point the ranking is effectively decided by the remaining, lower-weighted stability and speed terms rather than by accuracy. The stability term *Stab* itself penalizes both an implausible discontinuity between the last observed value and the first forecast value, and a mismatch between the volatility of the forecast and the historical volatility of the series, both normalized by the historical standard deviation. The model with the lowest composite score S among successfully trained, numerically usable candidates is selected as the system’s headline forecast, with the full ranking retained for inspection, as reported in Section X.

VII. Explainable AI

A. Scope and Design Choice

Two complementary interpretability mechanisms are implemented in the system: native, model-intrinsic feature importance for the tree-based models, and a retrieval-augmented large-language-model layer that narrates the forecast in natural language. A design choice made explicit here is that the system does not compute post-hoc, model-agnostic attribution methods such as SHAP [15] or LIME [16]; these are widely used in the broader explainable-AI literature (Section II) and are a natural direction for future extension of the feature-attribution component (Section XII), but were not implemented in the evaluated system, and this paper reports only interpretability mechanisms that are actually present in the codebase.

B. Native Feature Importance

For the gradient boosting and random forest models described in Section VI-B, the training procedure produces a gain-based (XGBoost) or impurity-based (random forest) importance score for every engineered feature as a direct byproduct of tree construction, at no additional computational cost. The system retains the ten most important features for the winning model, when the winning model is one of these two families, and surfaces them to the user as a ranked, labeled bar chart. Because these importances are computed identically to how the trees themselves were built, they are a global explanation of which lag, rolling-statistic, calendar, or exogenous features the model relied upon in aggregate across the training data, rather than a local explanation of any single forecasted value.

C. Retrieval-Augmented Natural-Language Explanation

The second, and more novel, interpretability mechanism is a retrieval-augmented generation layer built around a general-purpose large language model. Two independent context sources are combined before any natural-language response is generated. First, a set of deterministic textual summaries are constructed directly and exclusively from the pipeline’s own computed values: per-model accuracy

and error metrics, the identity and score of the selected model, the forecast’s minimum, maximum, and average values, a trend classification computed from a fixed percentage threshold, and, where applicable, per-product or per-group sales rankings and detected exogenous-feature usage. Second, for qualitative or advisory questions (for example, “how can I reduce stockouts”), the system retrieves the most semantically similar passages from a curated corpus of thirteen authored documents covering retail-operations topics such as inventory classification, safety stock, promotional and markdown strategy, seasonal demand planning, and stockout-versus-overstock diagnosis, using cosine similarity between text embeddings stored in a local, persistent vector index. This corresponds to the retrieval-augmented generation pattern of Lewis et al. [19], in which language-model generation is conditioned on retrieved source text rather than relying solely on the model’s parametric knowledge, reducing the risk of the response fabricating claims unrelated to the retrieved evidence.

D. Global and Local Explanations

In the terminology of the explainability literature, the tree-based feature-importance mechanism provides a global explanation of model behavior, while the natural-language layer can operate at both levels: a proactive business insights report narrates a global summary of the forecast and dataset, whereas the conversational assistant can answer local questions about a specific number, model, or product in the current result. Numeric questions, such as “which product sold the most,” are answered exclusively from the deterministic, pre-computed per-product summary rather than from retrieval or from the language model’s own estimation, since retrieval-based grounding for a fact the system already knows exactly would only introduce additional hallucination risk rather than reduce it; retrieval is reserved for genuinely qualitative, advisory content.

E. Consistency Constraints and Trust

A specific and, in the authors’ assessment, important engineering measure is that the prompt supplied to the language model explicitly enumerates the system’s own deterministic labels, most notably the trend classification and its fixed percentage threshold, and instructs the model never to re-derive or contradict them in its own words, even when the underlying numeric change is negative and could colloquially be described as a decline outside the labeled threshold band. The prompt additionally states plainly which models currently do and do not use exogenous features such as price, promotions, or holidays for the specific forecast being discussed, so that the language model cannot imply a capability the system did not actually exercise on this run. These constraints exist because a natural-language explanation that quietly contradicts the numbers the system itself computed would be more

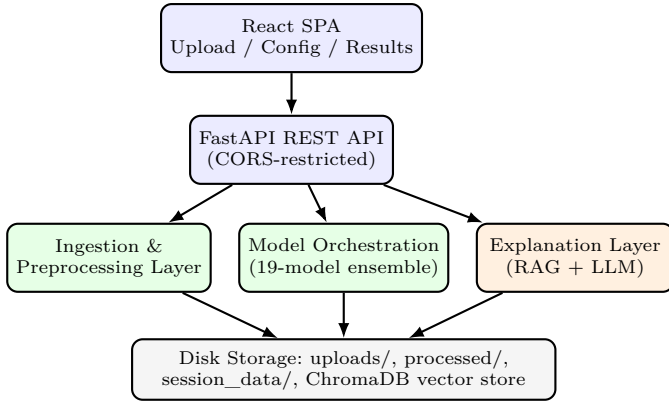


Fig. 1: Overall system architecture. The React frontend communicates exclusively with the FastAPI backend over a REST API; the backend’s three logical layers, ingestion/preprocessing, model orchestration, and explanation, all read from and write to a shared file-based storage layer rather than a database.

damaging to user trust than providing no explanation at all; grounding the language model in the pipeline’s own deterministic output, rather than allowing it to reason independently over the raw numbers, is the mechanism by which this consistency is enforced in practice.

F. How These Mechanisms Improve Trust

Feature importance answers the question of which inputs the winning model relied upon, while the grounded language layer answers the qualitatively different question of what the forecast means for the business and why a particular model was trusted, including an explicit statement of confidence derived from the model’s own cross-validated accuracy. Presenting both together, rather than either mechanism alone, allows a non-technical stakeholder to move from “the system predicts a number” to “the system predicts a number, driven primarily by these business factors, with this level of confidence, and here is what a comparable retail situation typically implies,” which is the practical bar this system was designed to meet.

VIII. System Architecture

A. Overall Architecture

Figure 1 depicts the overall system architecture. The React frontend communicates exclusively with the FastAPI backend through a REST API; the backend is organized into the ingestion/preprocessing, model orchestration, and explanation layers introduced in Section IV, all three of which read from and write to a shared, file-based storage layer rather than a relational or document database, as discussed in Section IV.

B. Backend Architecture

The backend is organized as a set of FastAPI routers, one each for upload, forecasting, per-group ranking, the

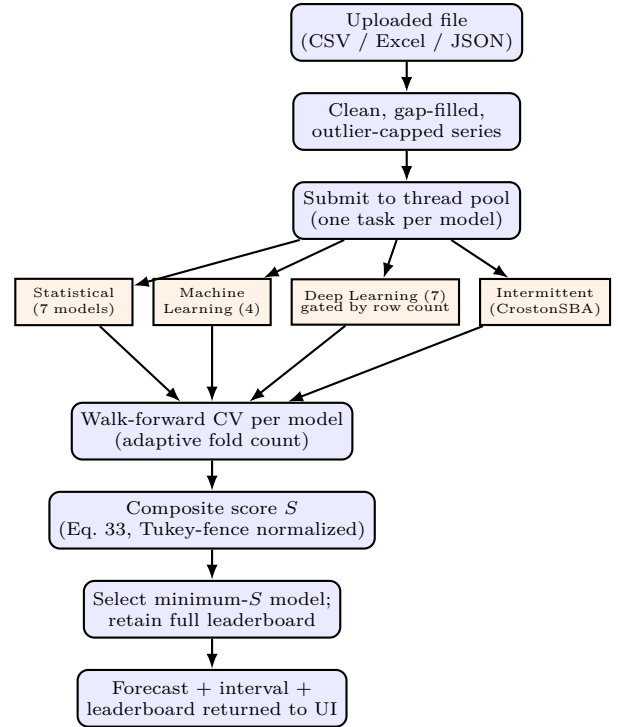


Fig. 2: Training and prediction pipeline. All nineteen registered models are submitted concurrently to a shared thread pool; deep learning models are withheld when the series falls below the frequency-specific minimum row count. Every surviving model is evaluated via walk-forward cross-validation and ranked by the composite score of Eq. (33).

conversational assistant, and the business insights report, backed by a set of stateless service modules that implement preprocessing, model training and orchestration, best-model selection, per-model architectures, retrieval-augmented generation, and disk-based session persistence. This separation allows the model orchestration logic to remain entirely agnostic to which specific models accept additional arguments such as a country code or exogenous features; such per-model variation is instead handled by constructing a request-scoped override of the shared model registry at the router level.

C. Machine Learning Pipeline and Prediction Flow

Figure 2 illustrates the training and prediction pipeline described in Section IV: the cleaned series is submitted to a shared thread pool that trains every applicable model concurrently, subject to the deep-learning row-count gate; every surviving model is evaluated by walk-forward cross-validation; and the composite score of Eq. (33) determines the selected model and the retained leaderboard.

D. Data Flow

Figure 3 details the automated data-cleaning flow applied to every uploaded dataset, exactly as executed for

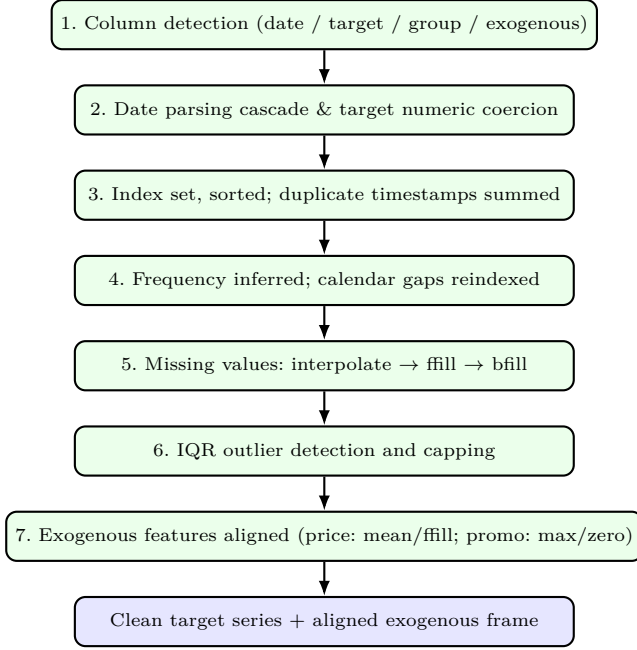


Fig. 3: Automated data preprocessing pipeline applied to every uploaded dataset prior to model training, exactly as executed for the evaluation dataset in Section V (Table II).

the evaluation dataset described in Section V.

E. Deployment Architecture

Figure 4 shows the deployment topology. The frontend is served as a static single-page application build, and the backend runs as an independent ASGI process; the two communicate over HTTP with cross-origin resource sharing restricted to an explicit allow-list, extensible for a production frontend origin via an environment variable. The only external network dependency in the entire architecture is the large-language-model API used exclusively by the explanation layer; the ingestion, preprocessing, and forecasting functionality of the system operates fully without external network access.

IX. Experimental Setup

A. Hardware

All experiments reported in this paper were executed on a single machine equipped with an Apple M3 Pro system-on-chip (ARM64 architecture) and 18 GB of unified memory, running macOS 15.7.3. No GPU acceleration was used; all deep learning training, where attempted, executed on CPU. This is noted explicitly because it directly affects the deep-learning training-time figures reported in Section X and should not be taken as representative of GPU-accelerated training throughput.

B. Software Environment

The backend is implemented in Python 3.14 using the library versions summarized in Table V. The clas-

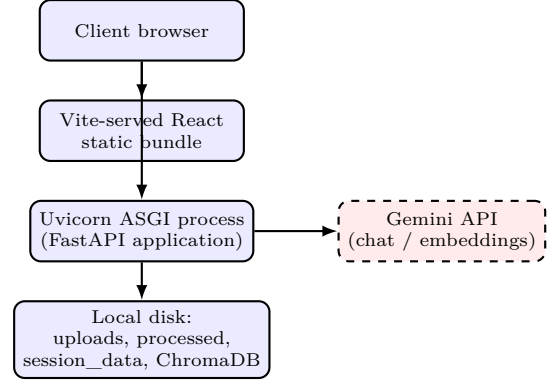


Fig. 4: Deployment architecture. The frontend and backend are independently deployable processes; CORS origins and the frontend’s backend base URL are environment-configured. The only external network dependency is the large-language-model API used by the explanation layer; core forecasting functions without it.

TABLE V: Technology Stack

Component	Technology	Version
Backend framework	FastAPI	0.136.0
ASGI server	Uvicorn	0.44.0
Data manipulation	pandas	3.0.2
Numerical computing	NumPy	2.4.4
Classical ML	scikit-learn	1.8.0
Gradient boosting	XGBoost	3.2.0
Statistical models	statsmodels	0.14.6
Additive regression	Prophet	1.3.0
Deep learning	PyTorch	2.11.0
LLM / embeddings	google-genai	2.12.1
Vector store	ChromaDB	1.5.9
Frontend framework	React	18.3
Build tool	Vite	6.x
Charting	Recharts	2.15

sical machine learning models (Section VI-B) are implemented using scikit-learn [36], and the statistical models (ARIMA, SARIMA, SARIMAX, and Holt-Winters exponential smoothing) are implemented using the statsmodels library [37]. The frontend is implemented in React 18 using Vite as the build tool and development server, Recharts for charting, Axios for HTTP communication, and Tailwind CSS for styling. The complete dependency set for both frontend and backend is pinned in version-controlled dependency manifests to ensure the reported results are reproducible against the exact library versions used.

C. Large Language Model Configuration

The explanation layer uses a general-purpose instruction-tuned language model (Gemini, accessed through the google-genai client library) for both conversational responses and structured report generation, and a corresponding embedding model for retrieval over

the retail-domain knowledge base. Report generation additionally constrains the model to return a fixed JSON schema (dataset summary, forecast summary, recommendations, revenue/trend signal, and stock-alert summary), so that the frontend can render the response deterministically rather than parsing free-form text.

D. Evaluation Protocol

The deployed system evaluates candidates by walk-forward cross-validation with the adaptive fold-count rule of Eq. (28), computing the metrics of Eqs. (29)–(32) on each fold and averaging. Selection then uses the composite score of Eq. (33). This paper deliberately does not adopt that protocol for measurement, because it cannot support the comparison being made.

Selecting the best of nineteen candidates by cross-validated error and then reporting that same cross-validated error is upward biased: the minimum of many noisy estimates is optimistic, and the bias grows with the number of candidates. Where models differ by only a few percent, as they do here, this bias can exceed the differences under interpretation. Every experiment reported in Section X therefore withholds a final test window of 28 observations from each series. Models are trained and cross-validated on the remainder, every selection policy chooses using only cross-validated quantities, and the chosen model’s forecast is then scored against the withheld window, which no part of the selection path has observed.

E. Error Measure

Realized error is reported as the mean absolute scaled error (MASE),

$$\text{MASE} = \frac{\frac{1}{h} \sum_{t=1}^h |y_t - \hat{y}_t|}{\frac{1}{n-m} \sum_{t=m+1}^n |y_t - y_{t-m}|}, \quad (35)$$

scaling absolute test error by the in-sample mean absolute error of a seasonal-naive forecast at period $m = 7$. RMSE cannot be averaged across series of differing scale, whereas MASE is scale-free, which is why the M-competitions adopt scaled errors for multi-series comparison [3]. Values below 1.0 indicate a forecast better than the seasonal-naive benchmark.

F. Statistical Comparison

Policies are compared across series using the Friedman test for the omnibus hypothesis, followed by pairwise Wilcoxon signed-rank tests with Holm correction across the policy family, the standard protocol for comparing methods over multiple datasets [38]. Comparisons are paired by construction, since every policy selects from the same candidate pool on the same series. Levels 3 and 5 comprise 10 and 7 series respectively—the complete populations of M5 stores and departments—and are reported descriptively without inferential claims.

G. Nested Selection Over Series

Comparing scoring configurations introduces the same bias one level higher: choosing the configuration that scores best across all series and reporting that score would select on the evaluation data. Configuration comparisons therefore partition the series into five folds, choose a configuration using only the training folds, and score it on the withheld fold. Reported figures estimate the performance of the selection procedure, not of the configuration that happens to rank first overall. Training-time budgets are treated as candidates within this same partition rather than read off a frontier computed over all series, so that a reported budget is one chosen without reference to the data on which it is scored.

H. Hyperparameter Selection

Hyperparameters for the statistical models (ARIMA/SARIMA/SARIMAX order, Holt-Winters trend/seasonal form, moving-average window) are selected automatically per dataset by grid search under cross-validation, as described in Section VI; the machine learning models’ regularization and neighbor-count hyperparameters (k for k -nearest neighbors; C and ϵ for support vector regression) are likewise cross-validated per dataset. The gradient boosting, random forest, and deep learning architectures use fixed hyperparameter configurations, reported in full in Section VI, that were established during development rather than tuned per dataset; per-dataset hyperparameter search for these models is identified as future work in Section XII.

I. Dataset Used for Evaluation

The experiments reported in Section X use the retail transaction dataset described in Section V, uploaded and processed through the deployed system’s own upload and forecast endpoints rather than through a separate offline evaluation script, so that the reported figures reflect the exact end-to-end behavior, including automated column detection and preprocessing, that a user of the system would experience.

X. Results

A. Experimental Scope and Completion

The evaluation covers 487 series drawn from five aggregation levels of the M5 hierarchy (Table VI). Two levels are structurally small: M5 contains exactly ten stores and seven departments, so levels 3 and 5 exhaust their populations rather than sample them and can never be enlarged. Results at those levels are therefore reported descriptively and no significance is claimed from them. Levels 9, 10, and 12 supply 70, 200, and 200 series respectively and carry the statistical weight of the study.

Every series contains 1,941 daily observations before the held-out window is removed, which places all of them above the system’s 500-observation minimum for deep learning at daily frequency. All nineteen models therefore

TABLE VI: M5 Aggregation Levels Evaluated

Level	Grouping	Series	Median CV	Powered
3	store	10	0.231	no
5	department	7	0.246	no
9	store $\times$ dept.	70	0.317	yes
10	item	200	0.531	yes
12	item $\times$ store	200	0.765	yes

CV denotes the coefficient of variation (standard deviation over mean) of the series, a scale-free measure of volatility. Levels 3 and 5 exhaust their populations—M5 contains exactly ten stores and seven departments—so they cannot be enlarged and no significance is claimed from them. Levels 10 and 12 are capped at 200 series sampled by descending volume.

trained on every series, the first configuration in which the deep learning family was exercised rather than gated out. At level 9, 1,330 of 1,330 model fits completed without failure or timeout.

B. Composite Scoring Versus Single-Metric Ranking

Table VII reports the central comparison. At every aggregation level, ranking candidates on a single cross-validated accuracy metric produces lower realized held-out error than the six-term composite score of Eq. (33). Where the comparison is adequately powered the margin is decisive: against cross-validated RMSE the composite loses 50 of 61 decided series at level 9 ($p_{\text{holm}} = 2.1 \times 10^{-7}$) and 117 of 169 at level 10 ($p_{\text{holm}} = 3.6 \times 10^{-6}$). At level 12 the direction is unchanged but the effect attenuates to marginal significance ($p_{\text{holm}} = 0.079$), which we report rather than omit; the policy spread compresses substantially at item–store granularity, where every method converges toward seasonal-naïve performance.

The result does not depend on which accuracy metric serves as the baseline. Selection on cross-validated MAE is comparable to, and by median marginally better than, selection on RMSE at every level; the composite loses to both. We retain RMSE as the reference baseline throughout because it is the conventional choice and because it is the metric the composite itself weights most heavily, but the finding should be read as concerning single-metric selection generally rather than RMSE in particular.

The composite is additionally not distinguishable from a seasonal-naïve forecast at level 9 ($p_{\text{holm}} = 0.149$) and is outperformed by it outright at level 12 (median 0.774 against 0.766). At no level does it beat the practice of always using one fixed model such as Prophet or ARIMA.

C. Which Term Actually Governs Selection

The composite assigns 80% of its nominal weight to accuracy terms (0.40 RMSE, 0.20 MAE, 0.20 MAPE) and only 0.10 to computational speed. Its behavior does not follow that intent. Decomposing each model’s score into per-term contributions and measuring, for every series, the spread each term induces across the candidate set gives the effective influence reported in Table VIII. The speed term accounts for between 63% and 88% of all

discriminating power, while RMSE—carrying four times the nominal weight—accounts for between 3.5% and 7.1%.

The cause is visible in the normalized ranges. On a representative level-9 series, normalized RMSE spans $[0.479, 1.032]$ while normalized speed spans $[3.6 \times 10^{-5}, 43.55]$, a ratio of 78.8 between the two spreads. Each term is divided by its own Tukey fence, $Q_3 + 1.5\text{IQR}$, which bounds the denominator but places no bound on the resulting ratio. Training time across the registry ranges from approximately 5×10^{-5} s for a moving average to 118 s for a grid-searched SARIMA—six orders of magnitude—whereas cross-validated RMSE across the same candidates differs by a factor of roughly 1.2. Because only three of nineteen models are slow, both quartiles of the training-time distribution fall among the fast models, the interquartile range is small, and the fence lands far below the actual cost of the expensive candidates.

This is not a subtle preference: the composite never selects SARIMA or SARIMAX at level 9, in any of the 70 series, despite those two models being the choice of RMSE-based selection in 57 of them and appearing among the oracle’s selections 21 times. Isolating why, term by term and series by series, the speed term is the single largest contributor to their exclusion in 100% of series, with a mean contribution gap of +3.52 against a countervailing RMSE advantage of only -0.012 —a ratio of roughly 300 to 1.

D. Correcting the Normalization

Two defects follow from the analysis above: the fence bounds denominators rather than ratios, and R^2 is the one term never normalized at all, entering the sum as a raw signed quantity alongside five scaled ones. We evaluated two corrections against the shipped scheme, holding weights, models, forecasts, and held-out data fixed so that only the scaling varies. The clipped scheme caps every fence-normalized term at 1.0 and places R^2 on the same footing; the balanced scheme additionally rescales each capped term across the candidate set so that every criterion spans exactly $[0, 1]$ and effective influence equals nominal weight by construction.

Both corrections behave exactly as designed—under balanced, measured effective shares match nominal weight shares to six decimal places—and neither materially improves realized accuracy. At level 9 the sequence is 1.324, 1.326, and 1.311 MASE for shipped, clipped, and balanced respectively, with the balanced scheme changing only 15 of 70 selections (Wilcoxon $p = 0.21$). Correcting the scaling is necessary for the score to mean what it claims, but it is not sufficient to make the score competitive.

E. Configuration Selection Under Nested Cross-Validation

Because no single correction sufficed, we compared a family of candidate scoring configurations: the shipped weights under both normalizations, three leave-one-criterion-out variants, an accuracy-only configuration dis-

TABLE VII: Realized Held-Out Error by Selection Policy and Aggregation Level (Median MASE; Lower is Better)

Selection policy	L3 ($n=10$)	L5 ($n=7$)	L9 ($n=70$)	L10 ($n=200$)	L12 ($n=200$)
Oracle (unachievable floor)	0.840	0.722	0.785	0.540	0.571
Cross-validated MAE only	0.930	0.960	0.892	0.661	0.676
Cross-validated RMSE only (baseline)	0.947	0.960	0.909	0.693	0.695
Always Prophet	0.973	0.895	0.949	0.743	0.695
Always ARIMA	1.126	1.198	1.068	0.681	0.692
Seasonal naive	0.973	1.087	1.051	0.752	0.766
Composite score (Eq. 33)	1.264	1.695	1.183	0.773	0.774
Naive (last value)	2.298	2.046	1.598	0.954	0.876
Composite vs. cross-validated RMSE, pairwise over series:					
Wins-losses	2–5	2–5	11–50	52–117	71–97
p_{holm}	1.00	1.00	2.1×10^{-7}	3.6×10^{-6}	0.079
Omnibus test across all policies:					
Friedman χ^2	44.6	26.9	295.2	—	—
p	1.1×10^{-6}	1.4×10^{-3}	2.7×10^{-58}	—	—

MASE below 1.0 indicates a forecast better than the in-sample seasonal-naive benchmark. Medians are reported because the per-series MASE distribution is right-skewed; bold marks the best achievable policy at each level, excluding the oracle. Cross-validated RMSE is retained as the comparison baseline throughout this paper because it is the conventional single-metric choice, but MAE-based selection is comparable or marginally better at every level—the substantive finding is that the composite loses to either single accuracy metric, not to RMSE specifically. Levels 3 and 5 are underpowered by construction ($n = 10$ and $n = 7$ exhaust the available stores and departments); their p -values are reported for completeness but support no inference. Pairwise comparisons use Wilcoxon signed-rank tests with Holm correction across the policy family [38].

TABLE VIII: Nominal Weight Versus Effective Influence on Model Selection

Term	Weight	Effective share		
		L9	L10	L12
Speed	0.10	0.883	0.751	0.631
R^2	−0.10	0.035	0.101	0.251
RMSE	0.40	0.035	0.071	0.054
MAE	0.20	0.019	0.039	0.028
MAPE	0.20	0.015	0.029	0.027
Stability	0.10	0.012	0.010	0.009

Effective share is the fraction of total score spread across candidate models attributable to each term, averaged over series. A term whose values are nearly identical across candidates cannot change the winner regardless of its weight; a term with large spread can decide the ranking even at small weight. Speed carries one quarter of RMSE’s nominal weight yet exerts between 12 and 25 times its influence. R^2 is the only term never divided by a normalization denominator, which is why its influence grows at finer granularity where R^2 varies most widely.

carding MAPE, speed, and stability, and—critically—twelve budget-constrained policies of the form “the most accurate model that trains within T seconds.” Selecting among these on all available series and reporting the winner would repeat precisely the error this study exists to characterize, so configurations are chosen within a five-fold partition over series and scored only on the fold withheld from that choice.

Table IX reports the outcome. A budget-constrained policy is selected at every level, and no weighted-sum configuration is ever selected at levels 10 or 12. At the two finest levels the selected policy beats single-metric ranking by approximately 0.040 MASE, winning in all five folds with a 95% confidence interval excluding zero; at level 12 this replicates across three random fold assign-

TABLE IX: Nested Five-Fold Selection Over Series (Mean MASE)

	L9	L10	L12
Selection procedure (MASE)	0.959	0.786	0.695
Cross-validated RMSE only	0.948	0.826	0.735
Oracle floor	0.823	0.618	0.575
Margin vs. baseline	−0.010	+0.040	+0.040
95% CI (lower)	−0.030	+0.020	+0.018
95% CI (upper)	+0.010	+0.061	+0.062
Folds won	0/5	5/5	5/5
Policy selected	budget	budget	budget
most frequent	30 s	1.5–2 s	1.5 s
Weighted-sum ever selected	1/5	0/5	0/5
Optimism if selected in sample	+0.136	+0.168	+0.119

Candidates comprise eight weighted-sum configurations and twelve budget-constrained policies. Within each fold the candidate is chosen using only the remaining folds and scored on the withheld one, so the reported figure estimates the selection procedure rather than the candidate that happens to look best overall. The final row gives the difference between that honest estimate and the best in-sample score, and exceeds every treatment effect in this study. At level 9 the selected 30-second budget exceeds the cost of every candidate (maximum 13.6 s) and is therefore equivalent to imposing no constraint.

ments (margins +0.040, +0.037, +0.040), with the same 1.5-second budget chosen in fourteen of fifteen folds. At level 9 the procedure instead selects a 30-second budget, which—since no candidate there costs more than 13.6 s—is unconstrained selection under another name, yielding a margin of exactly zero in four of five folds. The method thus declines to impose a constraint where none helps.

F. Cost as a Constraint Rather Than a Penalty

Table X reports realized error as a function of the training-time budget. The relationship is non-monotone

TABLE X: Accuracy–Latency Frontier at Level 12 ($n=200$)

Budget (s)	Mean MASE	Mean CV RMSE	Mean cost (s)
0.01	0.748	12.544	0.0002
0.10	0.745	12.513	0.004
0.50	0.745	12.006	0.277
1.00	0.716	11.839	0.519
2.00	0.698	11.724	0.925
5.00	0.717	11.112	1.980
10.00	0.733	10.781	3.383
30.00	0.736	10.576	7.358
∞	0.735	10.562	10.843

Mean CV RMSE—the quantity the selector minimizes—decreases monotonically as the budget relaxes, confirming the selection rule behaves correctly. Realized MASE does not: it reaches a minimum near 2 seconds and degrades thereafter. Admitting the models with the lowest cross-validated error worsens out-of-sample accuracy, so the budget operates as regularization against validation-fold overfitting. The minimum uses 8.5% of the computation required by unconstrained selection while achieving 5% lower error.

in held-out accuracy while remaining monotone in the selection objective: mean cross-validated RMSE of the selected model decreases uniformly as the budget relaxes, exactly as it must, yet realized MASE reaches a minimum at 1.5–2 seconds and worsens thereafter. Admitting the models with the lowest cross-validated error actively degrades out-of-sample performance. A compute budget functions here as regularization against selecting models that overfit their validation folds—an effect present at levels 10 and 12 and absent at level 9, consistent with finer-grained series being shorter, noisier, and more susceptible to such overfitting.

Compared at matched cost, the constrained formulation dominates the composite outright. Figures in this subsection are means, matching the frontier of Table X, rather than the medians of Table VII. At level 12 the composite achieves 0.769 mean MASE while consuming 4.00 seconds of training time for the model it selects; a 2-second budget achieves 0.698 at 0.93 seconds. The constrained policy is simultaneously 9% more accurate and 77% cheaper. It also beats unconstrained single-metric selection, which requires 10.84 seconds to reach 0.735. The same pattern holds at level 10 (0.782 at 0.93 s against the composite’s 0.898 at 2.82 s and single-metric’s 0.826 at 7.07 s).

G. The Cost of Selecting In Sample

Across all three powered levels, the configuration that appears best when scored on every series outperforms the honest nested estimate by 0.119 to 0.204 MASE. This optimism exceeds every treatment effect reported in this paper by roughly a factor of three to five. Had configurations been chosen and reported on the same series—the natural procedure absent explicit precautions—the resulting claims would have been artifacts of that choice rather than measurements of it.

H. Deep Learning Model Performance

This is the first configuration in which all seven deep learning architectures trained rather than being withheld by the row-count gate, every series having 1,913 training observations against a 500-observation threshold. They are nonetheless almost entirely absent from the oracle’s selections at levels 9 and 10, which are dominated by Prophet, SARIMAX, Holt–Winters, and SARIMA. Their appearance in composite-score selections at level 12 (85 of 200 selections) reflects not accuracy but cost: shorter, sparser series train quickly, so the speed term ceases to exclude them. The gating policy’s premise—that deep learning requires substantially more data than these datasets provide to be competitive—is supported by its own out-of-sample results.

XI. Discussion

A. Why Additive Combination Fails for Heterogeneous Criteria

The composite score’s failure is not attributable to a poor choice of weights, and this distinction matters because it determines what would repair it. A weighted sum expresses a fixed exchange rate between criteria: a unit of normalized error is declared equivalent to some quantity of normalized cost. That exchange rate is only meaningful if the normalized quantities are commensurable, and normalizing by a robust denominator does not make them so. The Tukey fence bounds what is divided by; it says nothing about the magnitude of the resulting ratio.

Across this model registry, cross-validated RMSE varies between candidates by roughly 20%, because a family of reasonable forecasting methods applied to the same series produces broadly similar accuracy. Training time varies by six orders of magnitude, because the methods differ not in degree but in kind—a moving average performs a handful of arithmetic operations while a grid-searched SARIMA fits hundreds of candidate models. After division by their respective fences, the accuracy terms occupy a narrow band near unity while the cost term ranges over two decades. The stated 4:1 preference for accuracy over speed operates in practice as roughly 1:25 against it.

This generalizes beyond the specific system studied here. Any additive multi-criteria score combining an accuracy measure with a resource measure is exposed to the same failure whenever the resource distribution is heavy-tailed, which it typically is: computational cost across algorithm families is naturally multiplicative, while accuracy among viable algorithms is naturally bounded. The defect is not that the weights were chosen badly but that no fixed weight vector can be correct when the terms’ effective ranges differ by two orders of magnitude and vary with the dataset.

B. Constraints Rather Than Penalties

Expressing a resource requirement as a constraint avoids the problem entirely, because a budget is absolute where a penalty is relative. Under the composite score, a model taking two seconds is penalized severely when its competitors are instantaneous and barely at all when they are slow—the same model receives different treatment depending on the company it keeps. A budget states a requirement the deployment actually has, admits every candidate meeting it, and optimizes accuracy exactly within that set. The resulting accuracy–latency frontier is directly interpretable by an operator choosing an operating point, which an implied exchange rate is not.

The empirical result is stronger than the argument requires. At the two finest aggregation levels the constrained formulation is not merely a better articulation of the same trade-off; it dominates the composite on both axes simultaneously, achieving lower error at lower cost, and it also outperforms unconstrained single-metric selection. The last of these was unexpected and is the study’s most substantive positive finding.

C. Compute Budgets as Regularization

The frontier’s non-monotonicity in realized error, against strict monotonicity in the selection objective, admits a clean interpretation. Relaxing the budget can only enlarge the candidate set, so the minimum achievable cross-validated error can only fall—and it does, uniformly. That held-out error nevertheless rises past a threshold means the additional candidates admitted at higher budgets achieve their lower validation error partly by fitting the validation folds rather than the underlying process. Excluding them by cost is a form of capacity control that happens to correlate with the mechanism producing the overfitting: the expensive models in this registry are expensive precisely because they search large hyperparameter spaces, and search over a validation criterion is the standard route to overfitting it.

That the effect is present at levels 10 and 12 and absent at level 9 supports rather than complicates this reading. Finer aggregation yields sparser, noisier series in which validation-fold error is a weaker estimate of generalization error, so selection on that estimate is more easily corrupted and a constraint that limits search has more to correct. Where series are well-aggregated and validation error is reliable, the procedure correctly declines to constrain, selecting a budget exceeding every candidate’s cost.

D. Selection Bias as a Recurring Hazard

The same error appeared four times in the course of this work at successively higher levels of abstraction: selecting a model on cross-validation folds and reporting that fold error; tuning score weights on the evaluation set; selecting a scoring configuration on all available series; and, finally, reading an optimal budget off a frontier computed over the same series used to report it. Each instance is individually

easy to overlook, and each inflates the reported result in the same direction.

The measured magnitude justifies the concern quantitatively rather than in principle. In-sample configuration selection is optimistic by 0.119 to 0.204 MASE—between three and five times every treatment effect reported in this paper. A study of this design that omitted nesting would produce claims determined by that bias rather than by the interventions it purports to measure. We therefore report the performance of the selection procedure rather than of any selected artifact, and treat budgets as candidates subject to the same discipline as scoring configurations.

E. Threats to Validity

Several considerations bound the interpretation of these results. The evaluation uses a single forecast origin per series at a fixed 28-step horizon; rolling-origin evaluation across multiple horizons would strengthen the estimates, though the direction of the principal comparison is consistent across five aggregation levels and three random fold assignments. All results derive from one dataset family, and while M5 is the standard retail forecasting benchmark, the specific magnitudes—particularly the 1.5–2-second budget optimum—should not be assumed transferable to registries with different cost profiles. The mechanism, being a statement about the relative spread of heterogeneous criteria, is expected to transfer more readily than the constants.

The effect sizes at level 10 warrant explicit caution: a margin of 0.040 MASE with a confidence interval spanning $[0.020, 0.061]$ is real but modest, and the interval derives from five folds. The negative result—that the composite score loses to single-metric ranking—rests on considerably firmer ground, with p -values of 10^{-6} or smaller at two independent levels.

Finally, the exogenous-feature capability described in Section IV is weakly exercised by this evaluation. M5 supplies no explicit promotion indicator, and a markdown proxy derived from price movements fires on only 21% of series at level 9 and 45% at level 12, with many items exhibiting monotonically non-decreasing prices. Claims regarding exogenous integration should be read as describing implemented capability rather than validated benefit.

F. Implications for System Design

The practical consequence for the system studied here was to replace the composite score with ranking on cross-validated RMSE, with an optional training-time budget disabled by default. The default is accuracy-only rather than budget-constrained because the useful budget proved dataset-dependent—beneficial at fine granularity, harmful at coarse—and cannot be determined from information available at request time. Accuracy-only ranking is the configuration that measured no worse than the alternative at every level examined, which makes it the appropriate

default; a budget is exposed for deployments where latency is a genuine requirement rather than an inferred preference.

More broadly, the finding argues against a common pattern in automated machine learning systems, in which several desirable properties are folded into a single scalar objective for convenience of implementation. Where the criteria are commensurable, that is reasonable. Where they differ in kind—accuracy against cost, quality against latency—separating the objective from the constraint set is both more defensible and, on this evidence, more accurate.

XII. Future Work

Several directions for extending the system are identified below. Two are noted explicitly because they are sometimes proposed as generic future-work items for forecasting systems but are, in fact, already implemented in the evaluated system: a transformer architecture is already one of the nineteen registered models (Section VI-C), and a large language model already underlies the explanation layer (Section VII). The corresponding future directions below therefore describe deepening or extending these existing capabilities rather than introducing them for the first time.

A. Real-Time and Streaming Forecasting

The current system operates in a batch mode, retraining models on a complete uploaded file per request. Extending the architecture to support incrementally updating forecasts as new observations arrive, without retraining every candidate model from scratch, would enable near-real-time dashboards and is a natural extension of the existing walk-forward evaluation procedure (Section VI-E), which already processes data in temporally ordered folds.

B. Cloud-Native and Horizontally Scaled Deployment

As noted in Section XI, the current file-based session and cache persistence layer is scoped to a single backend process. Migrating this state to a shared, network-accessible store would allow the backend to be deployed as multiple horizontally scaled instances behind a load balancer, and is a prerequisite for a cloud-native, auto-scaling deployment suitable for higher concurrent user volume than the current architecture targets.

C. Streaming Analytics Integration

Connecting the ingestion layer directly to a streaming data source, such as a point-of-sale event stream, rather than requiring a manually uploaded file, would allow the system to maintain a continuously updated forecast rather than one generated on demand, and would benefit from the real-time forecasting extension described above.

D. Additional and Pretrained Forecasting Transformer Variants

While a transformer encoder is already implemented (Section VI-C), it is trained from randomly initialized weights on each uploaded dataset independently. Recent pretrained, foundation-model-style time-series transformers, trained once on large heterogeneous time-series corpora and applied zero-shot or fine-tuned to a new series, represent a natural extension that could improve accuracy specifically on shorter series where training a transformer from scratch, as in the current implementation, is disadvantaged relative to statistical baselines.

E. Deeper Large Language Model Integration

The current explanation layer (Section VII) uses a general-purpose language model constrained by prompt-level instructions to remain consistent with the system’s deterministic outputs. An agentic extension, in which the language model can autonomously invoke system tools, for example to request an alternative forecast horizon, a different grouping, or a specific model’s leaderboard entry, in response to a user’s question, rather than being limited to the context assembled ahead of time by the backend, would extend the conversational assistant’s capability without weakening the existing grounding constraints, provided any tool invocation remains restricted to read-only, deterministic system operations.

F. Reinforcement Learning for Adaptive Model Weighting

Rather than a fixed composite scoring formula (Eq. (33)), a reinforcement-learning or bandit-based approach could adaptively learn per-dataset-category weightings for the score’s component metrics based on observed downstream forecast-error feedback over time, potentially improving on the fixed weighting used in the present system while retaining the same auditable set of components.

G. Automated Machine Learning and Hyperparameter Tuning

As identified in Section XI, the gradient boosting, random forest, and deep learning models currently use fixed hyperparameters rather than per-dataset tuning. Integrating an automated hyperparameter optimization procedure, bounded by a computation budget consistent with the system’s existing per-model timeout mechanism, is a direct and comparatively low-risk extension.

H. MLOps: Monitoring and Retraining Pipelines

The present system trains models synchronously within a single request and does not persist model artifacts for reuse or monitor forecast accuracy against subsequently observed actual values. Introducing a model-versioning and monitoring pipeline, tracking realized forecast error over time and triggering retraining when accuracy degrades, would extend the system toward a production MLOps workflow.

I. Federated Learning Across Multiple Retailers

For deployments serving multiple retail organizations, a federated learning scheme in which model updates, rather than raw sales data, are shared across organizations could allow the system to benefit from cross-organization patterns, such as category-level seasonality, without centralizing commercially sensitive transaction data.

J. Edge Deployment

Packaging a reduced subset of the lighter-weight statistical and classical machine learning models, which Section X shows train in well under a second, for on-device or edge deployment would allow forecasting to run within a retail location’s own point-of-sale or inventory-management hardware without a network dependency on the current server-based architecture, particularly relevant for the large-language-model explanation layer’s external API dependency discussed in Section VIII.

XIII. Conclusion

This paper evaluated a multi-criteria composite score for automated forecasting model selection against held-out data and found that it does not perform its intended function. Across 487 series drawn from five aggregation levels of the M5 hierarchy, ranking candidate models by cross-validated RMSE alone produced lower realized error than the six-term composite at every level, decisively so where the comparison was adequately powered ($p = 2.1 \times 10^{-7}$ and $p = 3.6 \times 10^{-6}$ at two independent levels). At its worst the composite was statistically indistinguishable from a seasonal-naive forecast.

The cause is structural rather than a matter of weight calibration. Normalizing each criterion by its own Tukey fence bounds the denominator but not the resulting ratio, so a training-time term spanning six orders of magnitude produced normalized values reaching 43 while accuracy terms, spanning a factor of 1.2, remained near unity. A criterion carrying a nominal weight of 0.10 consequently governed between 63% and 88% of selections, systematically excluding the classical statistical models that generalized best. Correcting the normalization so that effective influence matches nominal weight is necessary for the score to mean what it claims, but it does not make the score competitive: no weighted-sum configuration examined outperformed single-metric ranking.

Re-expressing the same requirement as a constraint does. Selecting the most accurate model that trains within a fixed budget outperformed single-metric selection by approximately 0.040 MASE at the two finest aggregation levels, winning in all five folds with confidence intervals excluding zero and replicating across random seeds, while consuming roughly an order of magnitude less computation. Under nested selection over series, a budget-constrained policy was chosen at every level and no weighted-sum configuration was chosen at either fine-grained level. Where no constraint was warranted the

procedure selected a budget exceeding every candidate’s cost, degenerating correctly to unconstrained selection.

A secondary finding concerns evaluation methodology. Selecting a scoring configuration on the same series used to report it inflates apparent performance by 0.119 to 0.204 MASE—three to five times any effect measured here—so nested cross-validation over series is a precondition for claims of this kind rather than a refinement of them. The same bias arose at four distinct levels of abstraction during this work, which suggests it is a structural hazard of automated selection systems rather than an incidental oversight.

The contribution is therefore twofold: a negative result establishing that additive combination of heterogeneous criteria fails when their effective ranges differ by orders of magnitude, with the mechanism identified and quantified; and a validated alternative in which cost is expressed as a constraint on the candidate set rather than a term in the objective. The system evaluated here was modified accordingly, replacing the composite score with accuracy-based ranking and an optional, disabled-by-default training-time budget. We suggest that automated machine learning systems combining quality and resource criteria in a single scalar objective should verify, rather than assume, that the resulting influence matches the intended one.

References

- [1] R. Fildes, S. Ma, and S. Kolassa, “Retail forecasting: Research and practice,” *International Journal of Forecasting*, vol. 38, no. 4, pp. 1283–1318, 2022.
- [2] J. D. Croston, “Forecasting and stock control for intermittent demands,” *Operational Research Quarterly*, vol. 23, no. 3, pp. 289–303, 1972.
- [3] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, “The M4 competition: 100,000 time series and 61 forecasting methods,” *International Journal of Forecasting*, vol. 36, no. 1, pp. 54–74, 2020.
- [4] —, “M5 accuracy competition: Results, findings, and conclusions,” *International Journal of Forecasting*, vol. 38, no. 4, pp. 1346–1364, 2022.
- [5] G. E. P. Box and G. M. Jenkins, *Time Series Analysis: Forecasting and Control*. San Francisco, CA: Holden-Day, 1970.
- [6] P. R. Winters, “Forecasting sales by exponentially weighted moving averages,” *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.
- [7] S. J. Taylor and B. Letham, “Forecasting at scale,” *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [8] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [9] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [10] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [11] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using RNN encoder-decoder for statistical machine translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.
- [12] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.

- [13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems* (NeurIPS), vol. 30, 2017.
- [14] B. Lim and S. Zohren, "Time-series forecasting with deep learning: A survey," *Philosophical Transactions of the Royal Society A*, vol. 379, no. 2194, p. 20200209, 2021.
- [15] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems* (NeurIPS), vol. 30, 2017.
- [16] M. T. Ribeiro, S. Singh, and C. Guestrin, "why should I trust you?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.
- [17] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., "Language models are few-shot learners," *Advances in Neural Information Processing Systems* (NeurIPS), vol. 33, pp. 1877–1901, 2020.
- [18] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2019, pp. 4171–4186.
- [19] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems* (NeurIPS), vol. 33, 2020, pp. 9459–9474.
- [20] J. K. Ord, R. Fildes, and N. Kourentzes, "Principles of business forecasting," 2nd Edition, Wessex Press Publishing Co., 2017.
- [21] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne, Australia: OTexts, 2018.
- [22] S. Makridakis, S. C. Wheelwright, and R. J. Hyndman, "Forecasting: Methods and applications," 3rd Edition, John Wiley & Sons, 1998.
- [23] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967.
- [24] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.
- [25] F. A. Gers, J. Schmidhuber, and F. Cummins, "Learning to forget: Continual prediction with LSTM," *Neural Computation*, vol. 12, no. 10, pp. 2451–2471, 2000.
- [26] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, 2021, pp. 11 106–11 115.
- [27] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [28] B. N. Oreshkin, D. Carpov, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis for interpretable time series forecasting," in *International Conference on Learning Representations (ICLR)*, 2020.
- [29] G. P. Zhang, "Time series forecasting using a hybrid ARIMA and neural network model," *Neurocomputing*, vol. 50, pp. 159–175, 2003.
- [30] S. Ma and R. Fildes, "Retail sales forecasting with meta-learning," in *European Journal of Operational Research*, vol. 288, no. 1, 2021, pp. 111–128.
- [31] A. A. Syntetos and J. E. Boylan, "The accuracy of intermittent demand estimates," *International Journal of Forecasting*, vol. 21, no. 2, pp. 303–314, 2005.
- [32] C. Bergmeir and J. M. Benítez, "On the use of cross-validation for time series predictor evaluation," *Information Sciences*, vol. 191, pp. 192–213, 2012.
- [33] T. Januschowski, J. Gasthaus, Y. Wang, D. Salinas, V. Flunkert, M. Bohlke-Schneider, and L. Callot, "Criteria for classifying forecasting methods," *International Journal of Forecasting*, vol. 36, no. 1, pp. 167–177, 2020.
- [34] F. Petropoulos, D. Apiletti, V. Assimakopoulos, M. Z. Babai, D. K. Barrow, S. Ben Taieb, C. Bergmeir, R. J. Bessa, J. Bijak, J. E. Boylan et al., "Forecasting: Theory and practice," *International Journal of Forecasting*, vol. 38, no. 3, pp. 705–871, 2022.
- [35] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems* (NeurIPS), vol. 32, 2019.
- [36] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [37] S. Seabold and J. Perktold, "Statsmodels: Econometric and statistical modeling with Python," in *Proceedings of the 9th Python in Science Conference (SciPy)*, 2010, pp. 92–96.
- [38] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.